PWM Signal Generation and Monitoring Project

November - 2024

Table of Contents

Abstract

This report presents the development and implementation of an embedded system for monitoring and controlling a PWM signal generated by a 555 timer, using an STM32F0 Discovery board. The system incorporates an optocoupler to control the frequency and duty cycle of the PWM signal, as well as several other features to enhance the user experience. The system allows for dynamic adjustments to the PWM signal's frequency by varying the voltage applied to the 555 timer, with real-time monitoring of the frequency and voltage on an OLED display. A button is implemented to toggle between displaying the PWM frequency and the control voltage on the OLED, providing users with an intuitive interface to track system parameters. Additionally, the system is capable of measuring the frequency of the generated signal, allowing users to observe how changes in voltage affect the timer’s frequency. This project successfully demonstrates the integration of microcontroller peripherals, ADCs, DACs, and SPI communication for precise control and monitoring of signal characteristics. The final implementation achieves the lab’s goal of monitoring, adjusting, and displaying the characteristics of a PWM signal while providing a flexible user interface for real-time feedback.

Introduction

Background

Pulse Width Modulation (PWM) is a widely used technique for controlling various systems, such as motor speed, LED brightness, and other analog parameters. The ability to monitor and adjust PWM signals in real-time is crucial in embedded system design, particularly for applications that require precise control over signal characteristics. The 555 timer, a versatile and reliable component, is commonly used to generate PWM signals, but it requires real-time monitoring and adjustments for effective operation. Monitoring PWM signals, especially in embedded systems, involves measuring the frequency and duty cycle of the signal and adjusting parameters like control voltage to maintain optimal performance. This project aims to demonstrate the practical use of the STM32F0 microcontroller in controlling and monitoring PWM signals with a user interface for real-time feedback.

Objective

The primary objective of this project is to design and implement an embedded system using the STM32F0 Discovery board to monitor and control a PWM signal generated by a 555 timer. The system allows for dynamic adjustments of the PWM frequency by varying the control voltage, while also measuring and displaying the frequency and voltage on an OLED display. A button interface is incorporated to toggle between displaying the frequency and control voltage, providing an intuitive and user-friendly means to observe real-time system behavior. Additionally, the system is designed to measure and display the effects of voltage changes on the PWM signal's frequency.

Overview of the Report

The report is structured as follows:

- System Specifications: This section outlines the technical specifications and components used in the project, including the STM32F0 Discovery board, the 555 timer, the optocoupler, and the OLED display.
- Design Approach: This section provides a detailed explanation of the system design, including the software and hardware architecture. It describes how the system was implemented, the role of various peripherals (ADC, DAC, GPIO, SPI), and how the control logic was realized.
- Experimental Setup: In this section, the hardware setup is explained in detail, including the wiring of the 555 timer, optocoupler, and potentiometer. The measurement setup is also discussed, detailing how the PWM signal frequency and potentiometer voltage are measured and displayed on the OLED screen.
- Results: The results section presents the observed outputs of the system, including the functionality of the PWM signal control and the user interface, with examples of how the frequency and voltage were displayed on the OLED.
- Analysis and Discussion: This section analyzes the system's performance, including potential limitations such as the range of frequencies achievable and the bottlenecks that may affect performance.
- Conclusion: The conclusion summarizes the key findings of the project and discusses potential improvements or extensions for future work.

System Specifications

Requirements

The system must fulfill both functional and non-functional requirements to ensure the correct monitoring and control of the PWM signal. The following outlines these requirements:

Functional Requirements:

- The system must measure and display the frequency and voltage of the PWM signal generated by the 555 timer.
- The system must allow the user to dynamically adjust the voltage via a potentiometer to observe the effect on the 555 timer's PWM frequency.
- The system must display the measured values (frequency and voltage) on the OLED display.
- The button must toggle between displaying the 555 timer's PWM signal and the external signal from the signal generator.
- The system must display both the frequency and voltage for both the 555 timer's signal and the signal generator on the OLED display when toggled.
- The system must allow the user to control the frequency and duty cycle of the PWM signal using the optocoupler, driven by the STM32F0's DAC.

Non-Functional Requirements:

- Responsiveness: The system should respond to user inputs (button presses) and voltage changes with minimal delay.
- User-Friendly Interface: The display and button controls should provide an intuitive and easy-to-use interface for the user.
- Reliability: The system must function reliably without crashes or errors during the operation of the PWM control and display features.

Components

The system consists of the following key components, each playing a vital role in achieving the system's functionality:

- STM32F0 Discovery Board: This is the central microcontroller used to monitor and control the PWM signal. It features various peripherals, including GPIO pins, ADCs, DACs, and interrupts, which are critical for the system's operation.
- 555 Timer IC: This timer is used to generate a PWM signal. Its frequency is controlled by the voltage applied to its control pin, which is adjusted using a potentiometer. The 555 timer's output is connected to the STM32F0 board, which will measure the frequency of the PWM signal.
- 4N35 Optocoupler: This component is used to isolate the control signal from the rest of the circuit while allowing the system to control the frequency of the PWM signal generated by the 555 timer. The STM32F0 will use its DAC to control the optocoupler, adjusting the frequency based on user input.
- OLED Display (SSD1306): This is used to display either the current frequency of the PWM signal or the potentiometer voltage, based on the user's input. It is controlled using the SPI communication protocol.
- Potentiometer: This is used to adjust the control voltage to the 555 timer. The potentiometer's voltage is read by the STM32F0 board via its ADC, and the adjusted value is used to control the frequency of the PWM signal.
- External Function Generator: In some configurations, an external function generator may be used to inject a reference PWM signal into the system, allowing the monitoring and analysis of system behavior under different input conditions.

Communication Protocols

Several communication protocols are used to facilitate the operation of the system:

- SPI (Serial Peripheral Interface): SPI is used to communicate with the SSD1306 OLED display. The STM32F0 sends commands and data to control the display's pixels, enabling the real-time display of either the PWM frequency or the potentiometer voltage.
- ADC (Analog-to-Digital Converter): The ADC on the STM32F0 is used to measure the voltage of the potentiometer. This voltage controls the frequency of the PWM signal generated by the 555 timer. The ADC converts the analog signal from the potentiometer into a digital value that is processed by the microcontroller.
- DAC (Digital-to-Analog Converter): The DAC on the STM32F0 is used to generate a control voltage that adjusts the 555 timer's frequency. The digital value output by the DAC is converted into an analog signal that is fed into the 555 timer's control pin.

- Interrupts: Interrupts are used to handle user input from the button. When the button is pressed, an interrupt is triggered, causing the microcontroller to toggle the displayed information between the PWM frequency and the potentiometer voltage. This ensures real-time responsiveness to user interaction.

Design Approach

The diagram below shows a simple version of how the system is connected.

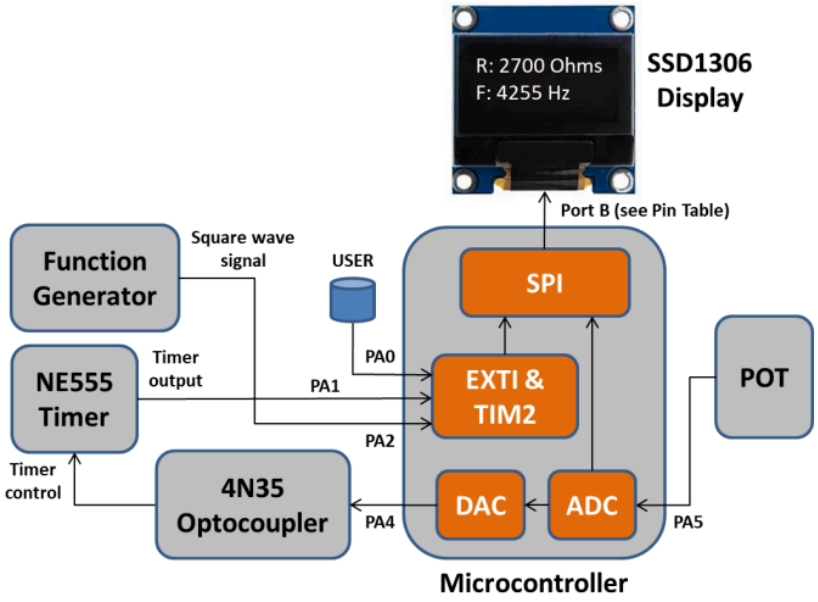

Figure 1. Diagram of Components and Microcontroller Connections [13].

The circuit diagram for 4N35 Optocoupler and NE555 Timer can be seen below in Figure 2. R1 and R2 are equal to 5000 ohms and C1 is equal to 0.1 uF.

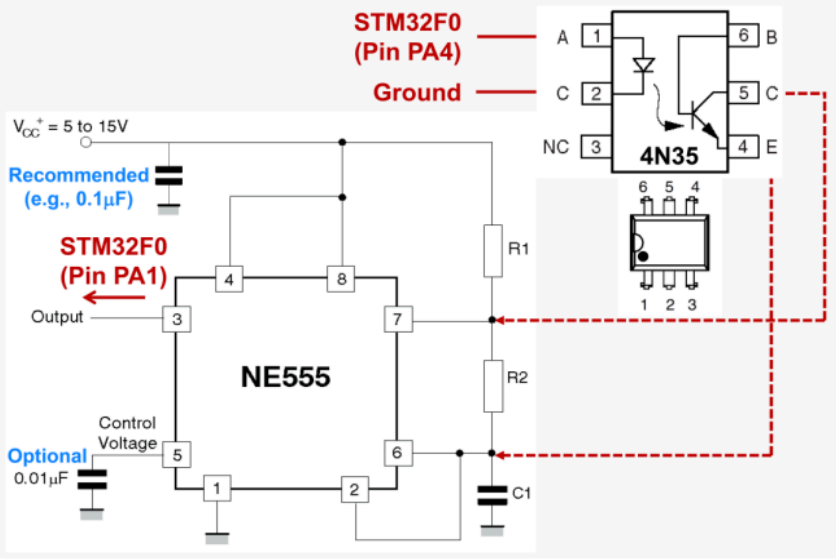

Figure 2. 4N35 Optocoupler and NE555 Timer Circuit Diagram [13].

Pin Configuration of STM32F0 Table

ADC: Set for continuous conversion to read the analog signal from potentiometer (PA5).

DAC: Configured to output an analog signal based on ADC input (PA4).

SPI: Configured for communication with the OLED display (PB3-PB7).

Interrupts: EXTI lines are mapped to handle rising-edge triggers from the user button (PA0), NE555 Timer (PA1), and Function Generator (PA2).

STM32F0	Signal	Mode
PA0	User Button	INPUT
PA1	555 Timer	INPUT
PA2	Function Generator	INPUT
PA4	DAC	ANALOG (OUTPUT)
PA5	ADC	ANALOG (INPUT)
PB3	SCLK	AF0 (OUTPUT), Pin 21 (J5)
PB4	RES	OUTPUT, Pin 19 (J5)

PB5	SDIN	AF0 (OUTPUT), Pin 17 (J5)
PB6	CS	OUTPUT, Pin 25 (J5)
PB7	D/C	OUTPUT, Pin 23 (J5)

Initial GPIOA Configuration Function

Code below configures GPIOA pins for input signals (User Button, 555 Timer, Function Generator) and analog modes for ADC and DAC.

```
void myGPIOA_Init()
{
    /* Enable clock for GPIOA peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
    /* Configure PA0, PA1, PA2 as input */
    GPIOA->MODER &= ~(GPIO_MODER_MODER0 | GPIO_MODER_MODER1 |
GPIO_MODER_MODER2);
    /* Ensure no pull-up/pull-down for PA0, PA1, PA2 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR0 | GPIO_PUPDR_PUPDR1 |
GPIO_PUPDR_PUPDR2);
    /* Configure Analog mode for PA4 and PA5*/
    GPIOA->MODER |= (GPIO_MODER_MODER4 | GPIO_MODER_MODER5);
    /* Ensure high-speed mode for PA4, PA5*/
    GPIOA->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR4 | GPIO_OSPEEDER_OSPEEDR5);
}
```

Initial GPIOB Configuration Function

Code below configures GPIOB pins for SPI communication and control signals for the OLED display

```
void myGPIOB_Init()
{
    /* Enable clock for GPIOB peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOBEN;
    /* Configure PB4, PB6, PB7 as output */
    GPIOB->MODER |= (GPIO_MODER_MODER4_0 | GPIO_MODER_MODER6_0 | GPIO_MODER_MODER7_0);
    /* Ensure push-pull mode selected for PB4, PB6, PB7*/
    GPIOB->OTYPER &= ~(GPIO_OTYPER_OT_4 | GPIO_OTYPER_OT_6 | GPIO_OTYPER_OT_7);
    /* Ensure high-speed mode for PB4, PB6, PB7*/
```

```
    GPIOB->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR4 | GPIO_OSPEEDER_OSPEEDR6 |
GPIO_OSPEEDER_OSPEEDR7);
    /* Ensure no pull-up/pull-down for PB4, PB6, PB7*/
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPDR4 | GPIO_PUPDR_PUPDR6 | GPIO_PUPDR_PUPDR7);
    /* Configure PB3 and PB5 as AF (default is AF0)*/
    GPIOB->MODER |= (GPIO_MODER_MODER3_1 | GPIO_MODER_MODER5_1);
}
```

Initial TIM2 Configuration Function

Code below configures TIM2 for frequency measurement.

```
void myTIM2_Init()
{
    /* Enable clock for TIM2 peripheral */
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
    /* Configure TIM2: buffer auto-reload, count up, stop on overflow,
    * enable update events, interrupt on overflow only */
    TIM2->CR1 = ((uint16_t)0x008c);
    /* Set clock prescaler value */
    TIM2->PSC = myTIM2_PRESCALER;
    /* Set auto-reloaded delay */
    TIM2->ARR = myTIM2_PERIOD;
    /* Update timer registers */
    TIM2->EGR = ((uint16_t)0x0001);
    /* Assign TIM2 interrupt priority = 0 in NVIC */
    NVIC_SetPriority(TIM2_IRQn, 0);
    /* Enable TIM2 interrupts in NVIC */
    NVIC_EnableIRQ(TIM2_IRQn);
    /* Enable update interrupt generation */
    TIM2->DIER |= TIM_DIER_UIE;
    /* counting timer pulses */
    TIM2->CR1 |= TIM_CR1_CEN;
}
```

Initial EXTI Configuration Function

Code below configures EXTI lines for rising-edge triggers on input pins (PA0, PA1, PA2) and manages interrupt priorities (EXTI0_1 with highest priority and EXTI1 is disabled to begin with).

```
void myEXTI_Init()
```

```
{
    /* Map EXTI line to PA0, PA1, PA2*/
    SYSCFG->EXTICR[0] &= 0x0000F000;
    /* EXTI0, EXTI1, EXTI2 line interrupts: set rising-edge trigger */
    EXTI->RTSR |= (EXTI_RTSR_TR0 | EXTI_RTSR_TR1 | EXTI_RTSR_TR2);
    /* Unmask interrupts from EXTI0, EXTI2 line */
    EXTI->IMR |= (EXTI_IMR_MR0 | EXTI_IMR_MR2);
    /* Assign EXTI1 interrupt priority = 0 in NVIC */
    NVIC_SetPriority(EXTI0_1_IRQn, 0);
    /* Assign EXTI2 interrupt priority = 64 in NVIC */
    NVIC_SetPriority(EXTI2_3_IRQn, 64);
    /* Enable EXTI2 interrupts in NVIC */
    NVIC_EnableIRQ(EXTI2_3_IRQn);
    /* Enable EXTI2 interrupts in NVIC */
    NVIC_EnableIRQ(EXTI0_1_IRQn);
}
```

Initial ADC Configuration Function

Code below configures ADC for continuous signal sampling on channel 5.

```
void myADC_Init()
{
    //Enable ADC Clock
    RCC->APB2ENR |= RCC_APB2ENR_ADCEN;
    //set continuous mode
    ADC1->CFGR1 |= ADC_CFGR1_CONT;
    //set overrun mode
    ADC1->CFGR1 |= ADC_CFGR1_OVRMOD;
    //set to channel 5
    ADC1->CHSELR |= ADC_CHSELR_CHSEL5;
    //set sample rate to 111 (239 clock cycles)
    ADC1->SMPR |= ADC_SMPR_SMP;
    //enable ADC
    ADC1->CR |= ADC_CR_ADEN;
    //wait for ADRDY to be 1, so it can start conversion process
    while((ADC1->ISR & ADC_ISR_ADRDY) == 0){}
    //start adc
    ADC1->CR |= ADC_CR_ADSTART;
```

```
}
```

Initial DAC Configuration Function

Code below configures DAC to channel 4.

```
void myDAC_Init()
{
    //Enable DAC clock
    RCC->APB1ENR |= RCC_APB1ENR_DACEN;
    //enable bit to channel 4 (default)
    DAC->CR |= DAC_CR_EN1;
}
```

EXTI0_1 IRQ Handler

Code below manages frequency measurements based on rising-edge detection for EXTI0 (User Button) and EXTI1 (555 Timer) inputs. If the User Button is pressed, then the system switches to EXTI2 detection (Function Generator), and if pressed again then it switches back to EXTI1 detection (555 Timer).

```
void EXTI0_1_IRQHandler()
{
    uint32_t counter;
    double period;
    double frequency;

    if ((EXTI->PR & EXTI_PR_PR1) != 0) {

    /* This code section is for measuring frequency on
    EXTI1 when inSig = 0 */
    if (inSig == 0){

        if(timerTriggered == 0){            //first rising edge
            timerTriggered = 1;
            counter = 0;
            TIM2->CNT= ((uint16_t)0x0000);  //clear count
            TIM2->CR1 |= TIM_CR1_CEN;       //start timer
        }else{                              //second rising edge
```

```
            timerTriggered = 0;
            TIM2->CR1 &= ~(TIM_CR1_CEN);    //stop timer
            counter = TIM2->CNT;
            period = ((double)counter/SystemCoreClock);
            frequency = 1/period;
            Freq = frequency;   //global
        }

        }

        EXTI->PR |= EXTI_PR_PR1;            //clear flag
    }


    if ((EXTI->PR & EXTI_PR_PR0) != 0) {

    /* If inSig = 0, then: let inSig = 1, disable
    EXTI1 interrupts, enable EXTI2 interrupts */
    /* Else: let inSig = 0, disable EXTI2 interrupts,
    enable EXTI1 interrupts */
        if(inSig == 0){
            inSig = 1;
            /* mask interrupts from EXTI1 line (Disable EXTI1)*/
            EXTI->IMR &= ~(EXTI_IMR_MR1);
            /* Enable EXTI2 interrupts in NVIC */
            NVIC_EnableIRQ(EXTI2_3_IRQn);
            timerTriggered = 0;
        }else{
            inSig = 0;
            /* Disable EXTI2 interrupts in NVIC*/
            NVIC_DisableIRQ(EXTI2_3_IRQn);
            /* unmask interrupts from EXTI1 line (Enable EXTI1)*/
            EXTI->IMR |= EXTI_IMR_MR1;
            timerTriggered = 0;
        }

        EXTI->PR |= EXTI_PR_PR0;            //clear flag
    }

}
```

EXTI2_3 IRQ Handler

Code below manages frequency measurements based on rising-edge detection for EXTI2 (Function Generator).

```
void EXTI2_3_IRQHandler()
{
    // Declare/initialize your local variables here...
    uint32_t counter;
    double period;
    double frequency;

    /* Check if EXTI2 interrupt pending flag is indeed set */
    if ((EXTI->PR & EXTI_PR_PR2) != 0){

        if(timerTriggered == 0){            //first rising edge
            timerTriggered = 1;
            counter = 0;
            TIM2->CNT= ((uint16_t)0x0000);  //clear count
            TIM2->CR1 |= TIM_CR1_CEN;       //start timer

        }else{                              //second rising edge
            timerTriggered = 0;
            TIM2->CR1 &= ~(TIM_CR1_CEN);    //stop timer
            counter = TIM2->CNT;
            period = ((double)counter/SystemCoreClock);
            frequency = 1/period;
            Freq = frequency; // global
        }
        EXTI->PR |= EXTI_PR_PR2;            //clear flag
    }
}
```

OLED Write Function

Code below is used to write to SPI1.

```
void oled_Write( unsigned char Value )
{
```

```
    /* Wait until SPI1 is ready for writing (TXE = 1 in SPI1_SR) */
    while ((SPI1->SR & SPI_SR_TXE) == 0) {}

    /* Send one 8-bit character:
       - This function also sets BIDIOE = 1 in SPI1_CR1
    */
    HAL_SPI_Transmit( &SPI_Handle, &Value, 1, HAL_MAX_DELAY );

    /* Wait until transmission is complete (TXE = 1 in SPI1_SR) */
    while ((SPI1->SR & SPI_SR_TXE) == 0) {}
}
```

OLED Write Command Function

Code below is used to send commands to control where to display information.

```
void oled_Write_Cmd( unsigned char cmd )
{
    // make PB6 = CS# = 1
    GPIOB->ODR |= GPIO_ODR_6;
    // make PB7 = D/C# = 0
    GPIOB->ODR &= ~(GPIO_ODR_7);
    // make PB6 = CS# = 0
    GPIOB->ODR &= ~(GPIO_ODR_6);
    oled_Write( cmd );
    // make PB6 = CS# = 1
    GPIOB->ODR |= GPIO_ODR_6;
}
```

OLED Write Data Function

Code below is used to send data to be displayed on OLED.

```
void oled_Write_Data( unsigned char data )
{
    // make PB6 = CS# = 1
    GPIOB->ODR |= GPIO_ODR_6;
    // make PB7 = D/C# = 1
    GPIOB->ODR |= GPIO_ODR_7;
```

```
    // make PB6 = CS# = 0
    GPIOB->ODR &= ~(GPIO_ODR_6);
    oled_Write( data );
    // make PB6 = CS# = 1
    GPIOB->ODR |= GPIO_ODR_6;
}
```

Initial OLED Configuration Function

Code below configures the OLED display and clears the display.

```
void oled_config( void )
{

    //SPI1 clock enabled
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
    SPI_Handle.Instance = SPI1;
    SPI_Handle.Init.Direction = SPI_DIRECTION_1LINE;
    SPI_Handle.Init.Mode = SPI_MODE_MASTER;
    SPI_Handle.Init.DataSize = SPI_DATASIZE_8BIT;
    SPI_Handle.Init.CLKPolarity = SPI_POLARITY_LOW;
    SPI_Handle.Init.CLKPhase = SPI_PHASE_1EDGE;
    SPI_Handle.Init.NSS = SPI_NSS_SOFT;
    SPI_Handle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
    SPI_Handle.Init.FirstBit = SPI_FIRSTBIT_MSB;
    SPI_Handle.Init.CRCPolynomial = 7;

    // Initialize the SPI interface
    HAL_SPI_Init( &SPI_Handle );

    // Enable the SPI
    __HAL_SPI_ENABLE( &SPI_Handle );

    /* Reset LED Display (RES# = PB4):
       - make pin PB4 = 0, wait for a few ms
       - make pin PB4 = 1, wait for a few ms
    */
    GPIOB->ODR &= ~(GPIO_ODR_4);
    for(int i = 0; i < 1; i++){}
    GPIOB->ODR |= GPIO_ODR_4;
```

```
    for(int i = 0; i < 1; i++){}

    // Send initialization commands to LED Display
    for ( unsigned int i = 0; i < sizeof( oled_init_cmds ); i++ )
    {
        oled_Write_Cmd( oled_init_cmds[i] );
    }

    /* Fill LED Display data memory (GDDRAM) with zeros:
       - for each PAGE = 0, 1, ..., 7
           set starting SEG = 0
           call oled_Write_Data( 0x00 ) 128 times
    */

    for(int page = 0; page <= 7; page++){

        oled_Write_Cmd(0xB0 | page);    //for page = 0,1,2..
        oled_Write_Cmd(0x02);           //lower half SEG 2
        oled_Write_Cmd(0x12);           //higher half SEG 2

        for(int i = 0; i < 128; i++) {
            oled_Write_Data(0x00);  // Write zero to each segment
        }
    }
}
```

OLED Refresh Function

Code below formats and displays measured resistance and frequency values on the OLED using SPI communication.

```
void refresh_OLED( void )
{
    // Buffer size = at most 16 characters per PAGE + terminating '\0'
    unsigned char Buffer[17];

    snprintf( Buffer, sizeof( Buffer ), "R: %5u Ohms", Res );

    /* Buffer now contains your character ASCII codes for LED Display
```

```
   - select PAGE (LED Display line) and set starting SEG (column)
   - for each c = ASCII code = Buffer[0], Buffer[1], ...,
       send 8 bytes in Characters[c][0-7] to LED Display
*/

oled_Write_Cmd(0xB2);           //for page2
oled_Write_Cmd(0x08);           //lower half SEG 8
oled_Write_Cmd(0x10);           //higher half SEG 8

for (int i = 0; i < 14; i++) {
    int c = Buffer[i];  // Get ASCII code from buffer

    for (int j = 0; j <= 7; j++) {

        oled_Write_Data(Characters[c][j]);
    }
}

snprintf( Buffer, sizeof( Buffer ), "F: %5u Hz", Freq );

/* Buffer now contains your character ASCII codes for LED Display
   - select PAGE (LED Display line) and set starting SEG (column)
   - for each c = ASCII code = Buffer[0], Buffer[1], ...,
       send 8 bytes in Characters[c][0-7] to LED Display
*/

oled_Write_Cmd(0xB4);           //for page4
oled_Write_Cmd(0x08);           //lower half SEG 8
oled_Write_Cmd(0x10);           //higher half SEG 8


for (int i = 0; i < 14; i++) {
    int c = Buffer[i];  // Get ASCII code from buffer

    for (int j = 0; j <= 7; j++) {

        oled_Write_Data(Characters[c][j]);
    }
}
```

```
    /* Wait for ~100 ms */
    for(int i = 0; i < 1; i++){}
}
```

Results

System Performance

During testing, the system demonstrated consistent and reliable performance across different test cases. Key observations and data points from the testing phase are as follows:

- **Range of PWM Signal Frequencies:**
  - When using the 555 timer, the system was able to achieve a range of PWM signal frequencies from **800 Hz to 1300 Hz**. This range showed that the frequency was adjustable within expected parameters based on the input resistance from the potentiometer.
  - For the function generator, the system successfully displayed frequencies ranging from **1 Hz to 500kHz**. This confirmed that the system could handle external signal inputs across a broad frequency spectrum.
- **Resistance Control of the 555 Timer:**
  - The potentiometer was used to modify the 555 timer's frequency, showing an **inverse relationship** between the resistance applied and the resulting frequency output. This behavior was consistent with the expected characteristics of the 555 timer circuit, confirming that the system could dynamically adjust the PWM frequency as intended.
  - The **resistance range** measured by the potentiometer varied from **0 ohms to 5000 ohms**, but it was noted that the signal sometimes fluctuated by a few ohms due to natural electrical noise or slight instabilities in the potentiometer's output.
- **Function Generator Behavior:**
  - The system maintained the intended behavior where the **resistance from the potentiometer did not affect the frequency** of the external signal generated by the function generator. This was consistent with the design goal that only the 555 timer's frequency was impacted by the resistance input.
- **System Responsiveness:**
  - The system displayed a high level of responsiveness, showing real-time or near-real-time updates on the OLED display as the potentiometer was adjusted or when the input signal changed. This real-time feedback ensured that users could monitor and make adjustments with confidence.

Analysis and Discussion

This section discusses any problems that we had or if the results we got are expected.

Range of PWM Signal Frequencies (Function Generator):

$$Minimum\ Period = \frac{Max\ \#\ in\ TIM2\ Register}{SystemCoreClock} = \frac{2^{32}}{48000000} = 89.48\ s$$

$$Minimum\ Frequency = \frac{1}{Minimum\ Period} = \frac{1}{89.48} = 11.18\ mHz$$

The function generator minimum frequency is similar to the expected minimum frequency of 11.18 mHz.

In our setup, the maximum measurable frequency was found to be around 500kHz. This limitation arises due to the sampling rate and signal processing speed of the microcontroller coupled with the code project. Specifically, at frequencies above this threshold, the system's measurement circuitry (e.g., timers, input capture functions) is unable to accurately detect each rising edge of the square wave.As the frequency increases, the transition period between the high and low states of the square wave becomes too short for the system to resolve, resulting in missed edges and inaccurate frequency readings. This is in part due to the microcontroller's timer resolution, which limits its ability to handle high-frequency inputs at sufficient timing precisions.

Range of PWM Signal Frequencies (555 Timer):

$$Frequency = \frac{1.443}{(R1 + 2R2)C1}$$

As can be seen in Figure 2. The potentiometer value is parallel with R2, so we get this equation.

$$Frequency = \frac{1.443}{(5000 + 2(5000\ ||\ Potentiometer))0.1x10^{-6}}$$

When potentiometer is set to 0 (minimum) we get

$$Frequency = \frac{1.443}{(5000 + 2(5000))0.1x10^{-6}} = 962\ Hz$$

When potentiometer is set to 5000 (maximum) we get

$$Frequency = \frac{1.443}{(5000 + 2(5000||5000))0.1x10^{-6}}$$

$$Frequency = \frac{1.443}{(5000 + 2(2500))0.1x10^{-6}} = 1443\ Hz$$

The range of the 555 timer frequencies based on the resistance is from 800-1300 Hz which is similar to the expected range of 962-1443 Hz.

Resistance Control:

$$Resistance = \frac{ADC\ value \times Max\ Resistance\ value}{Max\ ADC\ value} = \frac{ADC\ value \times 5000}{4096}$$

The range of resistance of the potentiometer is 0-5000 ohms which is similar to the expected range that is calculated using the equation above.

Switch Button Behavior:

The button used to switch between displaying the 555 timer's signal and the function generator's signal functioned as intended, toggling between the two inputs. However, an occasional issue was observed where pressing the button caused it to switch twice in quick succession, resulting in the display returning to the same state. This anomaly, suspected to be due to a debounce problem, may stem from hardware limitations or the absence of a delay mechanism to prevent rapid consecutive presses. Addressing this issue could improve the reliability of the button press response.

Conclusion

The project successfully met its objectives by designing and implementing an embedded system capable of monitoring and controlling a PWM signal generated by a 555 timer. The system leveraged the STM32F0 Discovery board and an assortment of peripheral components, including an optocoupler, potentiometer, and an OLED display, to create a flexible and responsive signal monitoring tool.

The key outcomes of this project included the ability to measure and display the frequency and voltage of the PWM signal generated by the 555 timer, as well as the input signal from an external function generator. The system demonstrated a frequency range of 800 Hz to 1300 Hz when controlled by the 555 timer and a broader range from 1 Hz to 500 kHz when using the function generator. The potentiometer effectively modulated the resistance, influencing the 555

timer's frequency as intended, while the function generator's frequency remained unaffected, aligning with the design goals. The display provided real-time feedback on both signals, showcasing the system's responsiveness to user adjustments.

A significant finding was that the system performed with near real-time responsiveness, which was crucial for practical applications. However, the occasional double-switching of the display due to potential button debouncing highlighted a minor limitation that could be addressed in future iterations to enhance reliability.

This project contributes to the field of embedded systems by showcasing how real-time monitoring and control of PWM signals can be achieved using a relatively simple hardware setup. The combination of signal processing, ADC/DAC control, and user interface through the OLED display offers a powerful platform for further development and adaptation to more complex signal analysis applications.

References

[1] ECE, University of Victoria, "4N35 Optocoupler Datasheet," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/4n35.pdf.

[2] ECE, University of Victoria, "SSD1306 OLED Display Datasheet," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/SSD1306.pdf.

[3] ECE, University of Victoria, "555 Timer Datasheet," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/555timer.pdf.

[4] ECE, University of Victoria, "PBMCUSLK Schematic," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/PBMCUSLK_SCH_D_0-1.pdf.

[5] ECE, University of Victoria, "PBMCUSLK User Guide," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/PBMCUSLK_UG.pdf.

[6] STMicroelectronics, "STM32F0 Discovery Board," [Online]. Available: https://www.st.com/en/evaluation-tools/stm32f0discovery.html.

[7] ECE, University of Victoria, "STM32F0 Schematic," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/stm32f0_schematic_MB1034.pdf.

[8] ECE, University of Victoria, "STM32F051R8T6 Datasheet," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/stm32f051r8t6_datatsheet_DM00039193.pdf.

[9] ECE, University of Victoria, "STM32F0 Reference Manual," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/stm32f0RefManual.pdf.

[10] ECE, University of Victoria, "STM32F0 Programming Manual," [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/supplement/STM32F0xxxProgrammingManual_DM00051352.pdf.

[11] D.N.Rakhmatov, University of Victoria, “IO Examples,” [Online]. Available: https://www.ece.uvic.ca/~daler/courses/ece355/iox.pdf.

[12] D.N.Rakhmatov, University of Victoria, “Interface Examples,” [Online]. Available: https://www.ece.uvic.ca/~daler/courses/ece355/interfacex.pdf.

[13] B.Sirna and D.N.Rakhmatov, University of Victoria,”ECE 355: Microprocessor-Based Systems Laboratory Manual,” [Online]. Available: https://www.ece.uvic.ca/~ece355/lab/ECE355-LabManual-2023.pdf.

Appendices

```
//
// This file is part of the GNU ARM Eclipse distribution.
// Copyright (c) 2014 Liviu Ionescu.
//
// ----------------------------------------------------------------------
#include <stdio.h>
#include "diag/Trace.h"
#include <string.h>
#include "cmsis/cmsis_device.h"

// ----------------------------------------------------------------------
//
// STM32F0 led blink sample (trace via $(trace)).
//
// In debug configurations, demonstrate how to print a greeting message
// on the trace device. In release configurations the message is
// simply discarded.
//
// To demonstrate POSIX retargetting, reroute the STDOUT and STDERR to the
// trace device and display messages on both of them.
//
// Then demonstrates how to blink a led with 1Hz, using a
// continuous loop and SysTick delays.
//
// On DEBUG, the uptime in seconds is also displayed on the trace device.
//
// Trace support is enabled by adding the TRACE macro definition.
// By default the trace messages are forwarded to the $(trace) output,
// but can be rerouted to any device or completely suppressed, by
// changing the definitions required in system/src/diag/trace_impl.c
// (currently OS_USE_TRACE_ITM, OS_USE_TRACE_SEMIHOSTING_DEBUG/_STDOUT).
```

```
//
// The external clock frequency is specified as a preprocessor definition
// passed to the compiler via a command line option (see the 'C/C++ General' ->
// 'Paths and Symbols' -> the 'Symbols' tab, if you want to change it).
// The value selected during project creation was HSE_VALUE=48000000.
//
/// Note: The default clock settings take the user defined HSE_VALUE and try
// to reach the maximum possible system clock. For the default 8MHz input
// the result is guaranteed, but for other values it might not be possible,
// so please adjust the PLL settings in system/src/cmsis/system_stm32f0xx.c
//
// ----- main() ---------------------------------------------------------------
// Sample pragmas to cope with warnings. Please note the related line at
// the end of this function, used to pop the compiler diagnostics status.
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wunused-parameter"
#pragma GCC diagnostic ignored "-Wmissing-declarations"
#pragma GCC diagnostic ignored "-Wreturn-type"



/* Definitions of registers and their bits are
   given in system/include/cmsis/stm32f051x8.h */


/* Clock prescaler for TIM2 timer: no prescaling */
#define myTIM2_PRESCALER ((uint16_t)0x0000)
/* Maximum possible setting for overflow */
#define myTIM2_PERIOD ((uint32_t)0xFFFFFFFF)



uint32_t timerTriggered = 0;
uint32_t inSig = 1;     // start with EXTI2 = enabled
unsigned int Freq = 0;  // Example: measured frequency value (global variable)
unsigned int Res = 0;   // Example: measured resistance value (global variable)


void myGPIOA_Init(void);
void myGPIOB_Init(void);
void myADC_Init(void);
void myDAC_Init(void);
void myTIM2_Init(void);
void myEXTI_Init(void);

void oled_Write(unsigned char);
void oled_Write_Cmd(unsigned char);
void oled_Write_Data(unsigned char);
void oled_config(void);
void refresh_OLED(void);

SPI_HandleTypeDef SPI_Handle;


// LED Display initialization commands
unsigned char oled_init_cmds[] =
{
    0xAE,
    0x20, 0x00,
    0x40,
```

```
    0xA0 | 0x01,
    0xA8, 0x40 - 1,
    0xC0 | 0x08,
    0xD3, 0x00,
    0xDA, 0x32,
    0xD5, 0x80,
    0xD9, 0x22,
    0xDB, 0x30,
    0x81, 0xFF,
    0xA4,
    0xA6,
    0xAD, 0x30,
    0x8D, 0x10,
    0xAE | 0x01,
    0xC0,
    0xA0
};



//
// Character specifications for LED Display (1 row = 8 bytes = 1 ASCII character)
// Example: to display '4', retrieve 8 data bytes stored in Characters[52][X] row
//          (where X = 0, 1, ..., 7) and send them one by one to LED Display.
// Row number = character ASCII code (e.g., ASCII code of '4' is 0x34 = 52)
//
unsigned char Characters[][8] = {
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b01011111, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // !
```

```
{0b00000000, 0b00000111, 0b00000000, 0b00000111, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // "
{0b00010100, 0b01111111, 0b00010100, 0b01111111, 0b00010100,0b00000000, 0b00000000, 0b00000000},  // #
{0b00100100, 0b00101010, 0b01111111, 0b00101010, 0b00010010,0b00000000, 0b00000000, 0b00000000},  // $
{0b00100011, 0b00010011, 0b00001000, 0b01100100, 0b01100010,0b00000000, 0b00000000, 0b00000000},  // %
{0b00110110, 0b01001001, 0b01010101, 0b00100010, 0b01010000,0b00000000, 0b00000000, 0b00000000},  // &
{0b00000000, 0b00000101, 0b00000011, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // '
{0b00000000, 0b00011100, 0b00100010, 0b01000001, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // (
{0b00000000, 0b01000001, 0b00100010, 0b00011100, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // )
{0b00010100, 0b00001000, 0b00111110, 0b00001000, 0b00010100,0b00000000, 0b00000000, 0b00000000},  // *
{0b00001000, 0b00001000, 0b00111110, 0b00001000, 0b00001000,0b00000000, 0b00000000, 0b00000000},  // +
{0b00000000, 0b01010000, 0b00110000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // ,
{0b00001000, 0b00001000, 0b00001000, 0b00001000, 0b00001000,0b00000000, 0b00000000, 0b00000000},  // -
{0b00000000, 0b01100000, 0b01100000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // .
{0b00100000, 0b00010000, 0b00001000, 0b00000100, 0b00000010,0b00000000, 0b00000000, 0b00000000},  // /
{0b00111110, 0b01010001, 0b01001001, 0b01000101, 0b00111110,0b00000000, 0b00000000, 0b00000000},  // 0
{0b00000000, 0b01000010, 0b01111111, 0b01000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // 1
{0b01000010, 0b01100001, 0b01010001, 0b01001001, 0b01000110,0b00000000, 0b00000000, 0b00000000},  // 2
{0b00100001, 0b01000001, 0b01000101, 0b01001011, 0b00110001,0b00000000, 0b00000000, 0b00000000},  // 3
{0b00011000, 0b00010100, 0b00010010, 0b01111111, 0b00010000,0b00000000, 0b00000000, 0b00000000},  // 4
{0b00100111, 0b01000101, 0b01000101, 0b01000101, 0b00111001,0b00000000, 0b00000000, 0b00000000},  // 5
{0b00111100, 0b01001010, 0b01001001, 0b01001001, 0b00110000,0b00000000, 0b00000000, 0b00000000},  // 6
{0b00000011, 0b00000001, 0b01110001, 0b00001001, 0b00000111,0b00000000, 0b00000000, 0b00000000},  // 7
{0b00110110, 0b01001001, 0b01001001, 0b01001001, 0b00110110,0b00000000, 0b00000000, 0b00000000},  // 8
{0b00000110, 0b01001001, 0b01001001, 0b00101001, 0b00011110,0b00000000, 0b00000000, 0b00000000},  // 9
{0b00000000, 0b00110110, 0b00110110, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // :
{0b00000000, 0b01010110, 0b00110110, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // ;
{0b00001000, 0b00010100, 0b00100010, 0b01000001, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // <
{0b00010100, 0b00010100, 0b00010100, 0b00010100, 0b00010100,0b00000000, 0b00000000, 0b00000000},  // =
{0b00000000, 0b01000001, 0b00100010, 0b00010100, 0b00001000,0b00000000, 0b00000000, 0b00000000},  // >
{0b00000010, 0b00000001, 0b01010001, 0b00001001, 0b00000110,0b00000000, 0b00000000, 0b00000000},  // ?
{0b00110010, 0b01001001, 0b01111001, 0b01000001, 0b00111110,0b00000000, 0b00000000, 0b00000000},  // @
{0b01111110, 0b00010001, 0b00010001, 0b00010001, 0b01111110,0b00000000, 0b00000000, 0b00000000},  // A
{0b01111111, 0b01001001, 0b01001001, 0b01001001, 0b00110110,0b00000000, 0b00000000, 0b00000000},  // B
{0b00111110, 0b01000001, 0b01000001, 0b01000001, 0b00100010,0b00000000, 0b00000000, 0b00000000},  // C
{0b01111111, 0b01000001, 0b01000001, 0b00100010, 0b00011100,0b00000000, 0b00000000, 0b00000000},  // D
{0b01111111, 0b01001001, 0b01001001, 0b01001001, 0b01000001,0b00000000, 0b00000000, 0b00000000},  // E
{0b01111111, 0b00001001, 0b00001001, 0b00001001, 0b00000001,0b00000000, 0b00000000, 0b00000000},  // F
{0b00111110, 0b01000001, 0b01001001, 0b01001001, 0b01111010,0b00000000, 0b00000000, 0b00000000},  // G
{0b01111111, 0b00001000, 0b00001000, 0b00001000, 0b01111111,0b00000000, 0b00000000, 0b00000000},  // H
{0b01000000, 0b01000001, 0b01111111, 0b01000001, 0b01000000,0b00000000, 0b00000000, 0b00000000},  // I
{0b00100000, 0b01000000, 0b01000001, 0b00111111, 0b00000001,0b00000000, 0b00000000, 0b00000000},  // J
{0b01111111, 0b00001000, 0b00010100, 0b00100010, 0b01000001,0b00000000, 0b00000000, 0b00000000},  // K
{0b01111111, 0b01000000, 0b01000000, 0b01000000, 0b01000000,0b00000000, 0b00000000, 0b00000000},  // L
{0b01111111, 0b00000010, 0b00001100, 0b00000010, 0b01111111,0b00000000, 0b00000000, 0b00000000},  // M
{0b01111111, 0b00000100, 0b00001000, 0b00010000, 0b01111111,0b00000000, 0b00000000, 0b00000000},  // N
{0b00111110, 0b01000001, 0b01000001, 0b01000001, 0b00111110,0b00000000, 0b00000000, 0b00000000},  // O
{0b01111111, 0b00001001, 0b00001001, 0b00001001, 0b00000110,0b00000000, 0b00000000, 0b00000000},  // P
{0b00111110, 0b01000001, 0b01010001, 0b00100001, 0b01011110,0b00000000, 0b00000000, 0b00000000},  // Q
{0b01111111, 0b00001001, 0b00011001, 0b00101001, 0b01000110,0b00000000, 0b00000000, 0b00000000},  // R
{0b01000110, 0b01001001, 0b01001001, 0b01001001, 0b00110001,0b00000000, 0b00000000, 0b00000000},  // S
{0b00000001, 0b00000001, 0b01111111, 0b00000001, 0b00000001,0b00000000, 0b00000000, 0b00000000},  // T
{0b00111111, 0b01000000, 0b01000000, 0b01000000, 0b00111111,0b00000000, 0b00000000, 0b00000000},  // U
{0b00011111, 0b00100000, 0b01000000, 0b00100000, 0b00011111,0b00000000, 0b00000000, 0b00000000},  // V
{0b00111111, 0b01000000, 0b00111000, 0b01000000, 0b00111111,0b00000000, 0b00000000, 0b00000000},  // W
{0b01100011, 0b00010100, 0b00001000, 0b00010100, 0b01100011,0b00000000, 0b00000000, 0b00000000},  // X
{0b00000111, 0b00001000, 0b01110000, 0b00001000, 0b00000111,0b00000000, 0b00000000, 0b00000000},  // Y
{0b01100001, 0b01010001, 0b01001001, 0b01000101, 0b01000011,0b00000000, 0b00000000, 0b00000000},  // Z
{0b01111111, 0b01000001, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // [
{0b00010101, 0b00010110, 0b01111100, 0b00010110, 0b00010101,0b00000000, 0b00000000, 0b00000000},  // back slash
{0b00000000, 0b00000000, 0b00000000, 0b01000001, 0b01111111,0b00000000, 0b00000000, 0b00000000},  // ]
```

```
    {0b00000100, 0b00000010, 0b00000001, 0b00000010, 0b00000100,0b00000000, 0b00000000, 0b00000000},  // ^
    {0b01000000, 0b01000000, 0b01000000, 0b01000000, 0b01000000,0b00000000, 0b00000000, 0b00000000},  // _
    {0b00000000, 0b00000001, 0b00000010, 0b00000100, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // `
    {0b00100000, 0b01010100, 0b01010100, 0b01010100, 0b01111000,0b00000000, 0b00000000, 0b00000000},  // a
    {0b01111111, 0b01001000, 0b01000100, 0b01000100, 0b00111000,0b00000000, 0b00000000, 0b00000000},  // b
    {0b00111000, 0b01000100, 0b01000100, 0b01000100, 0b00100000,0b00000000, 0b00000000, 0b00000000},  // c
    {0b00111000, 0b01000100, 0b01000100, 0b01001000, 0b01111111,0b00000000, 0b00000000, 0b00000000},  // d
    {0b00111000, 0b01010100, 0b01010100, 0b01010100, 0b00011000,0b00000000, 0b00000000, 0b00000000},  // e
    {0b00001000, 0b01111110, 0b00001001, 0b00000001, 0b00000010,0b00000000, 0b00000000, 0b00000000},  // f
    {0b00001100, 0b01010010, 0b01010010, 0b01010010, 0b00111110,0b00000000, 0b00000000, 0b00000000},  // g
    {0b01111111, 0b00001000, 0b00000100, 0b00000100, 0b01111000,0b00000000, 0b00000000, 0b00000000},  // h
    {0b00000000, 0b01000100, 0b01111101, 0b01000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // i
    {0b00100000, 0b01000000, 0b01000100, 0b00111101, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // j
    {0b01111111, 0b00010000, 0b00101000, 0b01000100, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // k
    {0b00000000, 0b01000001, 0b01111111, 0b01000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // l
    {0b01111100, 0b00000100, 0b00011000, 0b00000100, 0b01111000,0b00000000, 0b00000000, 0b00000000},  // m
    {0b01111100, 0b00001000, 0b00000100, 0b00000100, 0b01111000,0b00000000, 0b00000000, 0b00000000},  // n
    {0b00111000, 0b01000100, 0b01000100, 0b01000100, 0b00111000,0b00000000, 0b00000000, 0b00000000},  // o
    {0b01111100, 0b00010100, 0b00010100, 0b00010100, 0b00001000,0b00000000, 0b00000000, 0b00000000},  // p
    {0b00001000, 0b00010100, 0b00010100, 0b00011000, 0b01111100,0b00000000, 0b00000000, 0b00000000},  // q
    {0b01111100, 0b00001000, 0b00000100, 0b00000100, 0b00001000,0b00000000, 0b00000000, 0b00000000},  // r
    {0b01001000, 0b01010100, 0b01010100, 0b01010100, 0b00100000,0b00000000, 0b00000000, 0b00000000},  // s
    {0b00000100, 0b00111111, 0b01000100, 0b01000000, 0b00100000,0b00000000, 0b00000000, 0b00000000},  // t
    {0b00111100, 0b01000000, 0b01000000, 0b00100000, 0b01111100,0b00000000, 0b00000000, 0b00000000},  // u
    {0b00011100, 0b00100000, 0b01000000, 0b00100000, 0b00011100,0b00000000, 0b00000000, 0b00000000},  // v
    {0b00111100, 0b01000000, 0b00111000, 0b01000000, 0b00111100,0b00000000, 0b00000000, 0b00000000},  // w
    {0b01000100, 0b00101000, 0b00010000, 0b00101000, 0b01000100,0b00000000, 0b00000000, 0b00000000},  // x
    {0b00001100, 0b01010000, 0b01010000, 0b01010000, 0b00111100,0b00000000, 0b00000000, 0b00000000},  // y
    {0b01000100, 0b01100100, 0b01010100, 0b01001100, 0b01000100,0b00000000, 0b00000000, 0b00000000},  // z
    {0b00000000, 0b00001000, 0b00110110, 0b01000001, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // {
    {0b00000000, 0b00000000, 0b01111111, 0b00000000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // |
    {0b00000000, 0b01000001, 0b00110110, 0b00001000, 0b00000000,0b00000000, 0b00000000, 0b00000000},  // }
    {0b00001000, 0b00001000, 0b00101010, 0b00011100, 0b00001000,0b00000000, 0b00000000, 0b00000000},  // ~
    {0b00001000, 0b00011100, 0b00101010, 0b00001000, 0b00001000,0b00000000, 0b00000000, 0b00000000}   // <-
};


void SystemClock48MHz( void )
{
    // Disable the PLL
    RCC->CR &= ~(RCC_CR_PLLON);
    // Wait for the PLL to unlock
    while (( RCC->CR & RCC_CR_PLLRDY ) != 0 );
    // Configure the PLL for a 48MHz system clock
    RCC->CFGR = 0x00280000;
    // Enable the PLL
    RCC->CR |= RCC_CR_PLLON;
    // Wait for the PLL to lock
    while (( RCC->CR & RCC_CR_PLLRDY ) != RCC_CR_PLLRDY );
    // Switch the processor to the PLL clock source
    RCC->CFGR = ( RCC->CFGR & (~RCC_CFGR_SW_Msk)) | RCC_CFGR_SW_PLL;
    // Update the system with the new clock frequency
    SystemCoreClockUpdate();
}


void myGPIOA_Init()
{
    /* Enable clock for GPIOA peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
```

```
    /* Configure PA0, PA1, PA2 as input */
    GPIOA->MODER &= ~(GPIO_MODER_MODER0 | GPIO_MODER_MODER1 | GPIO_MODER_MODER2);
    /* Ensure no pull-up/pull-down for PA0, PA1, PA2 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR0 | GPIO_PUPDR_PUPDR1 | GPIO_PUPDR_PUPDR2);
    /* Configure Analog mode for PA4 and PA5*/
    GPIOA->MODER |= (GPIO_MODER_MODER4 | GPIO_MODER_MODER5);
    /* Ensure high-speed mode for PA4, PA5*/
    GPIOA->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR4 | GPIO_OSPEEDER_OSPEEDR5);
}


void myGPIOB_Init()
{
    /* Enable clock for GPIOB peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOBEN;
    /* Configure PB4, PB6, PB7 as output */
    GPIOB->MODER |= (GPIO_MODER_MODER4_0 | GPIO_MODER_MODER6_0 | GPIO_MODER_MODER7_0);
    /* Ensure push-pull mode selected for PB4, PB6, PB7*/
    GPIOB->OTYPER &= ~(GPIO_OTYPER_OT_4 | GPIO_OTYPER_OT_6 | GPIO_OTYPER_OT_7);
    /* Ensure high-speed mode for PB4, PB6, PB7*/
    GPIOB->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR4 | GPIO_OSPEEDER_OSPEEDR6 | GPIO_OSPEEDER_OSPEEDR7);
    /* Ensure no pull-up/pull-down for PB4, PB6, PB7*/
    GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPDR4 | GPIO_PUPDR_PUPDR6 | GPIO_PUPDR_PUPDR7);
    /* Configure PB3 and PB5 as AF (default is AF0)*/
    GPIOB->MODER |= (GPIO_MODER_MODER3_1 | GPIO_MODER_MODER5_1);
}


void myTIM2_Init()
{
    /* Enable clock for TIM2 peripheral */
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
    /* Configure TIM2: buffer auto-reload, count up, stop on overflow,
    * enable update events, interrupt on overflow only */
    TIM2->CR1 = ((uint16_t)0x008c);
    /* Set clock prescaler value */
    TIM2->PSC = myTIM2_PRESCALER;
    /* Set auto-reloaded delay */
    TIM2->ARR = myTIM2_PERIOD;
    /* Update timer registers */
    TIM2->EGR = ((uint16_t)0x0001);
    /* Assign TIM2 interrupt priority = 0 in NVIC */
    NVIC_SetPriority(TIM2_IRQn, 0);
    /* Enable TIM2 interrupts in NVIC */
    NVIC_EnableIRQ(TIM2_IRQn);
    /* Enable update interrupt generation */
    TIM2->DIER |= TIM_DIER_UIE;
    /* counting timer pulses */
    TIM2->CR1 |= TIM_CR1_CEN;
}



void myEXTI_Init()
{
    /* Map EXTI line to PA0, PA1, PA2*/
    SYSCFG->EXTICR[0] &= 0x0000F000;
    /* EXTI0, EXTI1, EXTI2 line interrupts: set rising-edge trigger */
    EXTI->RTSR |= (EXTI_RTSR_TR0 | EXTI_RTSR_TR1 | EXTI_RTSR_TR2);
    /* Unmask interrupts from EXTI0, EXTI2 line */
```

```
    EXTI->IMR |= (EXTI_IMR_MR0 | EXTI_IMR_MR2);
    /* Assign EXTI1 interrupt priority = 0 in NVIC */
    NVIC_SetPriority(EXTI0_1_IRQn, 0);
    /* Assign EXTI2 interrupt priority = 64 in NVIC */
    NVIC_SetPriority(EXTI2_3_IRQn, 64);
    /* Enable EXTI2 interrupts in NVIC */
    NVIC_EnableIRQ(EXTI2_3_IRQn);
    /* Enable EXTI2 interrupts in NVIC */
    NVIC_EnableIRQ(EXTI0_1_IRQn);
}


/* This handler is declared in system/src/cmsis/vectors_stm32f051x8.c */
void TIM2_IRQHandler()
{
    /* Check if update interrupt flag is indeed set */
    if ((TIM2->SR & TIM_SR_UIF) != 0)
    {
        trace_printf("\n*** Overflow! ***\n");
        /* Clear update interrupt flag */
        TIM2->SR &= ~(TIM_SR_UIF);
        /* Restart stopped timer */
        TIM2->CR1 |= TIM_CR1_CEN;
    }
}


void myADC_Init()
{
    //Enable ADC Clock
    RCC->APB2ENR |= RCC_APB2ENR_ADCEN;
    //set continuous mode
    ADC1->CFGR1 |= ADC_CFGR1_CONT;
    //set overrun mode
    ADC1->CFGR1 |= ADC_CFGR1_OVRMOD;
    //set to channel 5
    ADC1->CHSELR |= ADC_CHSELR_CHSEL5;
    //set sample rate to 111 (239 clock cycles)
    ADC1->SMPR |= ADC_SMPR_SMP;
    //enable ADC
    ADC1->CR |= ADC_CR_ADEN;
    //wait for ADRDY to be 1, so it can start conversion process
    while((ADC1->ISR & ADC_ISR_ADRDY) == 0){}
    //start adc
    ADC1->CR |= ADC_CR_ADSTART;
}


void myDAC_Init()
{
    //Enable DAC clock
    RCC->APB1ENR |= RCC_APB1ENR_DACEN;
    //enable bit to channel 4 (default)
    DAC->CR |= DAC_CR_EN1;
}


void EXTI0_1_IRQHandler()
{
    uint32_t counter;
```

```
    double period;
    double frequency;

    if ((EXTI->PR & EXTI_PR_PR1) != 0) {

    /* This code section is for measuring frequency on
    EXTI1 when inSig = 0 */
    if (inSig == 0){

        if(timerTriggered == 0){            //first rising edge
            timerTriggered = 1;
            counter = 0;
            TIM2->CNT= ((uint16_t)0x0000);  //clear count
            TIM2->CR1 |= TIM_CR1_CEN;       //start timer
        }else{                              //second rising edge
            timerTriggered = 0;
            TIM2->CR1 &= ~(TIM_CR1_CEN);    //stop timer
            counter = TIM2->CNT;
            period = ((double)counter/SystemCoreClock);
            frequency = 1/period;
            Freq = frequency;   //global
        }

        }

        EXTI->PR |= EXTI_PR_PR1;            //clear flag
    }


    if ((EXTI->PR & EXTI_PR_PR0) != 0) {

    /* If inSig = 0, then: let inSig = 1, disable
    EXTI1 interrupts, enable EXTI2 interrupts */
    /* Else: let inSig = 0, disable EXTI2 interrupts,
    enable EXTI1 interrupts */
        if(inSig == 0){
            inSig = 1;
            /* mask interrupts from EXTI1 line (Disable EXTI1)*/
            EXTI->IMR &= ~(EXTI_IMR_MR1);
            /* Enable EXTI2 interrupts in NVIC */
            NVIC_EnableIRQ(EXTI2_3_IRQn);
            timerTriggered = 0;
        }else{
            inSig = 0;
            /* Disable EXTI2 interrupts in NVIC*/
            NVIC_DisableIRQ(EXTI2_3_IRQn);
            /* unmask interrupts from EXTI1 line (Enable EXTI1)*/
            EXTI->IMR |= EXTI_IMR_MR1;
            timerTriggered = 0;
        }

        EXTI->PR |= EXTI_PR_PR0;            //clear flag
    }

}


/* This handler is declared in system/src/cmsis/vectors_stm32f051x8.c */
void EXTI2_3_IRQHandler()
{
```

```
    // Declare/initialize your local variables here...
    uint32_t counter;
    double period;
    double frequency;

    /* Check if EXTI2 interrupt pending flag is indeed set */
    if ((EXTI->PR & EXTI_PR_PR2) != 0){

        if(timerTriggered == 0){            //first rising edge
            timerTriggered = 1;
            counter = 0;
            TIM2->CNT= ((uint16_t)0x0000);  //clear count
            TIM2->CR1 |= TIM_CR1_CEN;       //start timer

        }else{                              //second rising edge
            timerTriggered = 0;
            TIM2->CR1 &= ~(TIM_CR1_CEN);    //stop timer
            counter = TIM2->CNT;
            period = ((double)counter/SystemCoreClock);
            frequency = 1/period;
            Freq = frequency; // global
        }
        EXTI->PR |= EXTI_PR_PR2;            //clear flag
    }
}


int main(int argc, char* argv[])
{
    SystemClock48MHz(); /* Initialize Clock*/
    myGPIOA_Init();     /* Initialize I/O port PA */
    myGPIOB_Init();     /* Initialize I/O port PB */
    myADC_Init();       /* Initialize ADC*/
    myDAC_Init();       /* Initialize DAC*/
    oled_config();      /* Initialize OLED*/
    myTIM2_Init();      /* Initialize timer TIM2 */
    myEXTI_Init();      /* Initialize EXTI */


    while (1)
    {
        while((ADC1->ISR & ADC_ISR_EOC) == 0){}   //wait for end of conversion
        uint32_t adc_val = ADC1->DR;
        Res = (adc_val*(double)5000)/(double)4095;               //calculate resistance
        DAC->DHR12R1 = adc_val;                                  //send adc val to DAC

        refresh_OLED();
    }
}


// LED Display Functions-------------------------------------------------
void refresh_OLED( void )
{
    // Buffer size = at most 16 characters per PAGE + terminating '\0'
    unsigned char Buffer[17];

    snprintf( Buffer, sizeof( Buffer ), "R: %5u Ohms", Res );

    /* Buffer now contains your character ASCII codes for LED Display
```

```
       - select PAGE (LED Display line) and set starting SEG (column)
       - for each c = ASCII code = Buffer[0], Buffer[1], ...,
           send 8 bytes in Characters[c][0-7] to LED Display
    */

    oled_Write_Cmd(0xB2);           //for page2
    oled_Write_Cmd(0x08);           //lower half SEG 8
    oled_Write_Cmd(0x10);           //higher half SEG 8

    for (int i = 0; i < 14; i++) {
        int c = Buffer[i];  // Get ASCII code from buffer

        for (int j = 0; j <= 7; j++) {

            oled_Write_Data(Characters[c][j]);
        }
    }

    snprintf( Buffer, sizeof( Buffer ), "F: %5u Hz", Freq );

    /* Buffer now contains your character ASCII codes for LED Display
       - select PAGE (LED Display line) and set starting SEG (column)
       - for each c = ASCII code = Buffer[0], Buffer[1], ...,
           send 8 bytes in Characters[c][0-7] to LED Display
    */

    oled_Write_Cmd(0xB4);           //for page4
    oled_Write_Cmd(0x08);           //lower half SEG 8
    oled_Write_Cmd(0x10);           //higher half SEG 8


    for (int i = 0; i < 14; i++) {
        int c = Buffer[i];  // Get ASCII code from buffer

        for (int j = 0; j <= 7; j++) {

            oled_Write_Data(Characters[c][j]);
        }
    }


    /* Wait for ~100 ms */
    for(int i = 0; i < 1; i++){}
}


void oled_Write_Cmd( unsigned char cmd )
{
    // make PB6 = CS# = 1
    GPIOB->ODR |= GPIO_ODR_6;
    // make PB7 = D/C# = 0
    GPIOB->ODR &= ~(GPIO_ODR_7);
    // make PB6 = CS# = 0
    GPIOB->ODR &= ~(GPIO_ODR_6);
    oled_Write( cmd );
    // make PB6 = CS# = 1
    GPIOB->ODR |= GPIO_ODR_6;
}
```

```
void oled_Write_Data( unsigned char data )
{
    // make PB6 = CS# = 1
    GPIOB->ODR |= GPIO_ODR_6;
    // make PB7 = D/C# = 1
    GPIOB->ODR |= GPIO_ODR_7;
    // make PB6 = CS# = 0
    GPIOB->ODR &= ~(GPIO_ODR_6);
    oled_Write( data );
    // make PB6 = CS# = 1
    GPIOB->ODR |= GPIO_ODR_6;
}


void oled_Write( unsigned char Value )
{
    /* Wait until SPI1 is ready for writing (TXE = 1 in SPI1_SR) */
    while ((SPI1->SR & SPI_SR_TXE) == 0) {}

    /* Send one 8-bit character:
       - This function also sets BIDIOE = 1 in SPI1_CR1
    */
    HAL_SPI_Transmit( &SPI_Handle, &Value, 1, HAL_MAX_DELAY );

    /* Wait until transmission is complete (TXE = 1 in SPI1_SR) */
    while ((SPI1->SR & SPI_SR_TXE) == 0) {}
}


void oled_config( void )
{

    //SPI1 clock enabled
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
    SPI_Handle.Instance = SPI1;
    SPI_Handle.Init.Direction = SPI_DIRECTION_1LINE;
    SPI_Handle.Init.Mode = SPI_MODE_MASTER;
    SPI_Handle.Init.DataSize = SPI_DATASIZE_8BIT;
    SPI_Handle.Init.CLKPolarity = SPI_POLARITY_LOW;
    SPI_Handle.Init.CLKPhase = SPI_PHASE_1EDGE;
    SPI_Handle.Init.NSS = SPI_NSS_SOFT;
    SPI_Handle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
    SPI_Handle.Init.FirstBit = SPI_FIRSTBIT_MSB;
    SPI_Handle.Init.CRCPolynomial = 7;


    // Initialize the SPI interface
    HAL_SPI_Init( &SPI_Handle );

    // Enable the SPI
    __HAL_SPI_ENABLE( &SPI_Handle );

    /* Reset LED Display (RES# = PB4):
       - make pin PB4 = 0, wait for a few ms
       - make pin PB4 = 1, wait for a few ms
    */
    GPIOB->ODR &= ~(GPIO_ODR_4);
    for(int i = 0; i < 1; i++){}
    GPIOB->ODR |= GPIO_ODR_4;
    for(int i = 0; i < 1; i++){}
```

```
    // Send initialization commands to LED Display
    for ( unsigned int i = 0; i < sizeof( oled_init_cmds ); i++ )
    {
        oled_Write_Cmd( oled_init_cmds[i] );
    }

    /* Fill LED Display data memory (GDDRAM) with zeros:
       - for each PAGE = 0, 1, ..., 7
           set starting SEG = 0
           call oled_Write_Data( 0x00 ) 128 times
    */

    for(int page = 0; page <= 7; page++){

        oled_Write_Cmd(0xB0 | page);   //for page = 0,1,2..
        oled_Write_Cmd(0x02);          //lower half SEG 2
        oled_Write_Cmd(0x12);          //higher half SEG 2

        for(int i = 0; i < 128; i++) {
            oled_Write_Data(0x00);  // Write zero to each segment
        }
    }
}

#pragma GCC diagnostic pop

// ----------------------------------------------------------------------------
```